

Taller 2: Búsqueda Local y Juegos Adversarios

Camilo Puerto Peña - 202211362

Daniela Munar González - 202311392

Nicolas Hurtado - 202414730

Andres Tello - 20222807

Punto 1: Configuración inicial mediante ascenso de colina

En este punto vamos a implementar `configuration_score()` y `hill_climbing()`. Para `configuration_score` nos dicen que `problem.score_components()` retorna la cobertura, la redundancia y la exposición en ese orden entonces es solo cuestión de guardarlos en variables y usar la formula:

puntaje = cobertura - redundancia - exposición

Y retornar el resultado.

```
7
8  def configuration_score(
9      problem: SmartGridOptimizationProblem, configuration: Configuration
10 ) -> float:
11     """
12     Combina cobertura, redundancia y exposición en un puntaje a maximizar.
13
14     Tips:
15     - Use problem.score_components(configuration); ya retorna cobertura,
16       redundancia y exposición en ese orden.
17     """
18     cobertura, reundancia, exposicion = problem.score_components(configuration)
19     return cobertura - reundancia - exposicion
```

`hill_climbing()` es un poco más elaborado, en esta función hay que retornar un objeto de clase `OptimizationResult` que tiene esta estructura:

- `best_configuration`: `Configuration`
- `best_score`: `float`
- `evaluations`: `int`
- `iterations`: `int`
- `history`: `list[Configuration]`
- `score_history`: `list[float]`

Durante el desarrollo de `hill_climbing()` hay que mantener estos valores para agregarlos al objeto.

Veamos la estructura general de la función:

1. Creamos `current`, una variable tipo `OptimizationResult` con una copia de `initial_configuration` como `best_configuration` y el puntaje de `initial_configuration` como `best_score`.
2. Creamos un `for` loop con `range(max_iterations)` y:

- a. Creamos una lista de vecinos “list_neighbors”
 - b. Definimos la variable “evals_per_cycle” donde guardaremos la cantidad de evaluaciones del ciclo
 - c. Creamos un for loop recorriendo “list_neighbors”
 - i. Buscamos el mejor vecino
 - ii. Si el mejor vecino es mejor que “current.best_score” lo guardamos en “max_neighbor_score”
 - iii. Cómo evaluamos el score del vecino, sumamos 1 a “evals_per_cycle”
 - d. Actualizamos current.evaluations y current.iterations
 - e. Si hay un vecino mejor, actualizamos el resto de current (best_score, best_configuration, history y score_history)
 - f. Si no, retornamos current
3. Llegamos al máximo de iteraciones entonces retornamos current

Representación de Solución	Una tupla binaria donde cada 1 representa un módulo en esa posición
Restricción de Validez	Cada configuración válida debe tener exactamente k módulos activos (debe haber k unos en la tupla)
Función Objetivo	Nuestra función objetivo es maximizar el puntaje calculado por $puntaje = cobertura - redundancia - exposición$
Relación de Vecindad	Una configuración es vecina de otra cuando mueves un módulo de una posición a otra. Por ejemplo si tenemos {100} algunos vecinos serán: {010} y {001}

Pruebas de resultados diferentes entre plateau_grid y local_peak:

```

Ejecutando:
/usr/local/bin/python /Users/andrestello/Documents/GitHub/taller2_IA/main.py -m optimization -a hillClimbing -l plateau_grid --seed 7 -n 5 -x 0.55 --iterations 500 -t

hillClimbing | mejor=94.00 | promedio=61.40 | desviación=19.63 | evaluaciones_promedio=2.0 | tiempo_promedio=0.000114s

=== CONFIGURACIÓN FINAL ===
#####
%0.C.N.C%
%......%
%N....O%
%.C.C..%
%N....N%
%......%
%C.O.C.N%
#####
Puntaje=94.00 | Evaluaciones=2 | Iteraciones=1

Ejecución finalizada correctamente.
Presiona Enter para volver al menú...

Ejecutando:
/usr/local/bin/python /Users/andrestello/Documents/GitHub/taller2_IA/main.py -m optimization -a hillClimbing -l local_peak --seed 7 -n 5 -x 0.55 --iterations 500 -t

hillClimbing | mejor=96.00 | promedio=70.00 | desviación=13.22 | evaluaciones_promedio=2.0 | tiempo_promedio=0.000119s

=== CONFIGURACIÓN FINAL ===
#####
%0..C..N..%
%..%...%
%N..C..O..%
%......%
%.N..C.N%
%..%...%
%C..O..N..%
#####
Puntaje=96.00 | Evaluaciones=2 | Iteraciones=1

Ejecución finalizada correctamente.
Presiona Enter para volver al menú...

```

En la imagen vemos que el algoritmo hill_climbing tiene resultados bastante diferentes en cada situación, cada una se repitió 5 veces: En plateau_grid tuvo un puntaje promedio de 61.40 contra una puntuación óptima de 94.00 con una desviación de 19.63. En las 5 repeticiones hizo 2 evaluaciones en promedio. En local_peak tuvo un promedio de 70.00 contra el mismo 96.00, además hizo las mismas 2 repeticiones en promedio y tuvo una desviación de 13.22.

Punto 2: Escape de óptimos locales mediante recocido simulado

En este punto buscamos implementar el algoritmo de recocido simulado para encontrar buenas configuraciones sin depender únicamente de mejoras inmediatas. La idea fue permitir que el algoritmo pudiera aceptar algunas soluciones peores de forma temporal, seguir explorando otras opciones y evitar quedarse atrapado rápidamente. Para esto usamos una temperatura que disminuye con cada iteración, vecinos seleccionados aleatoriamente y un registro separado entre la solución actual y la mejor solución encontrada.

2.1 cooling_schedule : En cooling_schedule calculamos cómo disminuye la temperatura durante la ejecución. initial_temperature representa la temperatura inicial, cooling_rate la velocidad de enfriamiento e iteration la iteración actual. Con estos valores aplicamos la fórmula $T(t) = T_0 * \alpha^t$ y retornamos la temperatura correspondiente. A medida que avanzan las iteraciones, la temperatura baja y el algoritmo va reduciendo la posibilidad de aceptar soluciones peores.

2.2 simulated_annealing : En simulated_annealing guardamos por separado la configuración actual y la mejor configuración encontrada. En cada iteración calculamos la temperatura, obtenemos los vecinos de la configuración actual y escogemos uno de forma aleatoria con rng.

Luego calculamos delta, que compara el puntaje del vecino con el puntaje actual. Si el vecino es mejor, lo aceptamos directamente; si es peor, usamos la probabilidad $\exp(\text{delta} / \text{temperature})$ para decidir si también puede ser aceptado.

Cuando una configuración es aceptada, pasa a ser la nueva configuración actual. Si además supera el mejor puntaje encontrado hasta ese momento, actualizamos `best_configuration` y `best_score`. También registramos el historial, las evaluaciones y las iteraciones realizadas. La ejecución termina cuando se alcanza la temperatura mínima o el límite de iteraciones, y finalmente retornamos el mejor resultado encontrado durante todo el proceso.

2.3 Configuraciones probadas y resultados:

Para observar el efecto de la temperatura inicial y del factor de enfriamiento, usamos el mismo layout `plateau_grid` y mantuvimos fijos la semilla, el número máximo de iteraciones y la cantidad de ejecuciones. Probamos cuatro combinaciones de `T0` y `alpha`, usando temperaturas iniciales de 10 y 30, junto con factores de enfriamiento de 0.90 y 0.99. En cada caso realizamos cinco ejecuciones para observar si los resultados se mantenían estables frente a la aleatoriedad.

```
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinela> # Prueba 1
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinela> python main.py -m optimization -a simulatedAnnealing -l plateau_grid --seed 7 -n 5 --iterations
750 --temperature 10 --cooling 0.90
simulatedAnnealing | mejor=94.00 | promedio=94.00 | desviación=0.00 | evaluaciones_promedio=220.0 | tiempo_promedio=0.008157s
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinela> # Prueba 2
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinela> python main.py -m optimization -a simulatedAnnealing -l plateau_grid --seed 7 -n 5 --iterations
750 --temperature 10 --cooling 0.99
simulatedAnnealing | mejor=94.00 | promedio=94.00 | desviación=0.00 | evaluaciones_promedio=751.0 | tiempo_promedio=0.028251s
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinela> # Prueba 3
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinela> python main.py -m optimization -a simulatedAnnealing -l plateau_grid --seed 7 -n 5 --iterations
750 --temperature 30 --cooling 0.90
simulatedAnnealing | mejor=94.00 | promedio=93.60 | desviación=0.80 | evaluaciones_promedio=230.0 | tiempo_promedio=0.009009s
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinela> # Prueba 4
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinela> python main.py -m optimization -a simulatedAnnealing -l plateau_grid --seed 7 -n 5 --iterations
750 --temperature 30 --cooling 0.99
simulatedAnnealing | mejor=94.00 | promedio=94.00 | desviación=0.00 | evaluaciones_promedio=751.0 | tiempo_promedio=0.030175s
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinela>
```

Los resultados muestran que las cuatro configuraciones lograron encontrar un mejor puntaje de 94. Sin embargo, hubo diferencias en el número de evaluaciones, el tiempo promedio y la estabilidad de los resultados. Las configuraciones con `alpha = 0.99` realizaron las 750 iteraciones y alcanzaron 751 evaluaciones promedio, mientras que con `alpha = 0.90` el algoritmo se detuvo antes porque la temperatura llegó más rápido al valor mínimo.

2.4 Análisis:

Al comparar las configuraciones vimos que con `alpha = 0.90` la temperatura baja más rápido, por eso el algoritmo termina antes y realiza menos evaluaciones. En las pruebas 1 y 3 se hicieron alrededor de 220 a 230 evaluaciones. En cambio, con `alpha = 0.99` la temperatura baja más lento, por eso el algoritmo siguió hasta completar las 750 iteraciones y realizó 751 evaluaciones. Esto también hizo que el tiempo de ejecución fuera mayor.

En los resultados, las cuatro configuraciones encontraron un mejor puntaje de 94. La prueba 3, con $T_0 = 30$ y $\alpha = 0.90$, tuvo un promedio de 93.60 y una desviación de 0.80, mientras que las otras pruebas tuvieron promedio de 94 y desviación de 0. En estas pruebas, usar $\alpha = 0.99$ hizo que el algoritmo buscara durante más tiempo, pero no encontró un puntaje mayor a 94.

2.5 Reflexión sobre el uso de IA

Usamos IA para entender cómo debía comportarse el recocido simulado frente al problema que debíamos resolver y para revisar que la lógica aplicada coincidiera con lo pedido en el taller. También la usamos para entender mejor algunos métodos de `SmartGridOptimizationProblem`, especialmente cómo obtener los vecinos y cómo aprovechar esa clase dentro de `simulated_annealing`.

Prompt usado:

Explicame cómo debería comportarse `simulated_annealing` para este problema y cómo puedo usar los métodos definidos en `problem.py`, especialmente `SmartGridOptimizationProblem`, basándote solo en el enunciado y en ese archivo.

Punto 3: Optimización poblacional mediante algoritmo genético

En este punto buscamos implementar un algoritmo genético para explorar varias configuraciones al mismo tiempo y encontrar una buena solución a lo largo de varias generaciones. La idea fue trabajar con una población de individuos, seleccionar los mejores, combinarlos mediante cruce, aplicar mutaciones y conservar algunos de los mejores resultados. Con este proceso buscamos que la población vaya mejorando generación tras generación y que el algoritmo guarde siempre la mejor configuración encontrada durante toda la ejecución.

3.1 one_point_crossover: En `one_point_crossover` tomamos dos padres y generamos dos hijos a partir de un punto de corte. Primero revisamos que ambos padres tengan la misma longitud y luego escogemos con `rng` una posición interna para hacer el cruce. El primer hijo toma la primera parte del padre 1 y la segunda parte del padre 2, mientras que el segundo hijo hace lo contrario. En esta función no reparamos los descendientes porque esa parte se hace después dentro del algoritmo genético.

3.2 swap_mutation: En `swap_mutation` decidimos primero si el individuo va a mutar usando `mutation_probability`. Si ocurre la mutación, buscamos las posiciones que tienen 1 y las que tienen 0, escogemos una de cada grupo con `rng` y las intercambiamos. Con esto cambiamos la posición de un módulo sin cambiar la cantidad total de módulos instalados. Si no ocurre la mutación, retornamos el individuo sin modificarlo.

3.3 genetic_algorithm: En genetic_algorithm comenzamos creando una población inicial y calculando el puntaje de cada individuo. Después guardamos el mejor encontrado y, en cada generación, conservamos los mejores individuos mediante elitismo. Para completar la nueva población seleccionamos padres por torneo, hacemos el cruce con one_point_crossover, reparamos los hijos y luego aplicamos swap_mutation. Este proceso continúa hasta completar nuevamente el tamaño de la población.

Después evaluamos la nueva generación y comparamos su mejor individuo con best_score. Si encontramos uno con mayor puntaje, actualizamos el mejor resultado global. También guardamos ese resultado en los historiales de cada generación. De esta forma no nos quedamos únicamente con el mejor individuo de la última generación, sino con el mejor que apareció durante toda la ejecución, como pide el taller.

3.4 Configuraciones probadas y resultados:

Para observar cómo cambian los resultados del algoritmo genético, usamos el mismo layout balanced_city y mantuvimos fija la semilla en 7 y la cantidad de ejecuciones en cinco. Probamos cuatro configuraciones diferentes, cambiando el tamaño de la población, la probabilidad de mutación y el número de generaciones. En la primera prueba usamos una población de 20 individuos, 100 generaciones y una mutación de 0.05; en la segunda aumentamos la población a 40; en la tercera mantuvimos la población en 40 y aumentamos la mutación a 0.20; y en la cuarta usamos 40 individuos, una mutación de 0.05 y aumentamos las generaciones a 200.

```
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinelas> # Prueba 1
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinelas> python main.py -m optimization -a geneticAlgorithm -l balanced_city --seed 7 -n 5 --population 20 --generations 100 --mutation 0.05 --elite 2
geneticAlgorithm | mejor=283.00 | promedio=281.00 | desviación=2.19 | evaluaciones_promedio=2020.0 | tiempo_promedio=0.062382s
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinelas> # Prueba 2
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinelas> python main.py -m optimization -a geneticAlgorithm -l balanced_city --seed 7 -n 5 --population 40 --generations 100 --mutation 0.05 --elite 2
geneticAlgorithm | mejor=283.00 | promedio=282.20 | desviación=0.98 | evaluaciones_promedio=4040.0 | tiempo_promedio=0.109741s
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinelas> # Prueba 3
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinelas> python main.py -m optimization -a geneticAlgorithm -l balanced_city --seed 7 -n 5 --population 40 --generations 100 --mutation 0.20 --elite 2
geneticAlgorithm | mejor=283.00 | promedio=281.40 | desviación=1.50 | evaluaciones_promedio=4040.0 | tiempo_promedio=0.131069s
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinelas> # Prueba 4
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinelas> python main.py -m optimization -a geneticAlgorithm -l balanced_city --seed 7 -n 5 --population 40 --generations 200 --mutation 0.05 --elite 2
geneticAlgorithm | mejor=283.00 | promedio=282.20 | desviación=0.98 | evaluaciones_promedio=8040.0 | tiempo_promedio=0.250816s
PS C:\Users\dany\Documents\U.ANDES\IA\RedCentinelas>
```

Los resultados muestran que las cuatro configuraciones alcanzaron un mejor puntaje de 283. La prueba 1 obtuvo un promedio de 281.00 con una desviación de 2.19, mientras que la prueba 2 obtuvo un promedio de 282.20 y una desviación de 0.98. En la prueba 3 el promedio fue de 281.40 con una desviación de 1.50, y en la prueba 4 el promedio volvió a ser 282.20 con una desviación de 0.98. También aumentaron las evaluaciones y el tiempo cuando se usó una población mayor o más generaciones, pasando de 2020 evaluaciones en la prueba 1 hasta 8040 en la prueba 4.

3.5 Análisis:

Al comparar las pruebas 1 y 2 vimos que aumentar la población de 20 a 40 individuos mantuvo el mejor puntaje en 283, pero el promedio subió de 281.00 a 282.20 y la desviación bajó de 2.19 a 0.98. Esto muestra que con una población más grande los resultados fueron más parecidos entre las ejecuciones, aunque también aumentaron las evaluaciones y el tiempo.

En la prueba 3 aumentamos la probabilidad de mutación de 0.05 a 0.20. El mejor puntaje siguió siendo 283, pero el promedio bajó a 281.40 y la desviación aumentó a 1.50. En la prueba 4 aumentamos las generaciones de 100 a 200 y el resultado promedio se mantuvo igual que en la prueba 2, pero las evaluaciones pasaron de 4040 a 8040 y el tiempo aumentó bastante. En estas pruebas, usar más generaciones no mejoró el puntaje final, pero sí hizo que el algoritmo trabajara durante más tiempo.

3.6 Reflexión sobre el uso de IA

Usamos IA para entender cómo aplicar el algoritmo genético al problema de optimización planteado en el taller, especialmente cómo representar y mejorar las configuraciones a lo largo de varias generaciones. También la usamos para comprender mejor los métodos de SmartGridOptimizationProblem y el manejo de la aleatoriedad con rng de random.Random, usando funciones como choice, randint y random.

Prompt usado:

Explicame cómo se debería comportar el algoritmo genético al problema de optimización del taller y cómo usar SmartGridOptimizationProblem junto con rng de random.Random, especialmente choice, randint y random, basándote solo en el enunciado, en problem.py y random.py.

Punto 4: Defensa adversaria mediante Minimax.

4.1 Tabla formulacion:

Componente	Descripcion
Estado inicial	Posición del defensor, posición del intruso y terminales críticas pendientes por activar.
Jugador en turno	Juego de 2 jugadores con turnos alternados, el defensor es el agente 0 (MAX, mueve

	primero) y el intruso es el agente 1 (MIN, mueve después)
Acciones	Cada agente en la cuadrícula puede moverse arriba, abajo, izquierda, derecha o permanecer quieto. No está permitido atravesar bloques ni salirse de los límites del mapa.
Función de resultado o sucesor	A partir de un estado y una acción se genera un nuevo estado mediante (state.generate_succesor).
Prueba de estado terminal	El juego termina cuando el defensor activa todas las terminales críticas o cuando el intruso ocupa la misma posición que el defensor.
Utilidad	Una victoria del defensor tiene utilidad +1000, mientras que una derrota tiene utilidad -1000

4.2 Implementación de minimax: En MinimaxAgent implementamos una búsqueda recursiva que evalúa las posibles acciones del defensor y del intruso. El defensor corresponde al agente 0 y busca maximizar el valor, mientras que el intruso corresponde a agente 1 y busca minimizarlo. En cada nivel generamos los estados sucesores con generate_successor() y alternamos entre los dos agentes hasta llegar a un estado terminal o alcanzar la profundidad indicada. Finalmente, retornamos la acción del defensor con el mayor valor minimax, conservando la primera acción si hay empate.

4.3 Modelado y criterio de desempate:

- **MAX(Defensor, Agente 0):** Le asignamos al defensor la función de maximización sobre los niveles correspondientes a los turnos en el árbol de juego. Su objetivo es elegir la acción que maximice el valor minimax del estado sucesor. Lo identificamos con la condición `is_max = agent_index == 0`.
- **MIN(Intruso, Agente 1):** Es el agente adverso en los niveles alternos, lo identificamos mediante `next_agent`. Su objetivo es minimizar el puntaje del defensor actualizando el estado con `value < best_value`.
- **Criterio de desempate:** La mejor acción se inicializa por defecto con la primera opción legal disponible (`best_action = actions[0]`). Cuando evaluamos los sucesores, usamos desigualdades estrictas “>” para MAX y “<” para MIN, por lo tanto, si una acción posterior genera exactamente el mismo valor minimax, la condición no se cumple y se mantiene la primera acción según el orden original entregado por `node.get_legal_actions(agent_index)`.

4.4 Tabla comparación: Evaluamos el agente minimax usando single_terminal como control basico y comparando profundidades 2 y 4 plies en tiny_defense y bottleneck.

	Profundidad (plies)	Accion Inicial	Resultado	Nodos Defensor	Nodos Intruso
single_terminal	2	East	victoria	112	72
tiny_defense	2	South	limite	954	837
tiny_defense	4	South	limite	16438	6755
bottleneck	2	Stop	derrota	27	19
bottleneck	4	West	victoria	2192	1470

```
PS C:\Users\nimau\Desktop\Escritorio\Clases 2026-2\IA\IA taller2\taller2_IA> py -3.11 main.py -m adversarial -a MinimaxAgent -l single_terminal -d 2 -q
MinimaxAgent | profundidad=2 plies | acción_inicial=East | resultado=victoria | finalización=activación completa | nodos=112 | nodos_intruso=72 | tiempo=0.004136s
PS C:\Users\nimau\Desktop\Escritorio\Clases 2026-2\IA\IA taller2\taller2_IA> py -3.11 main.py -m adversarial -a MinimaxAgent -l tiny_defense -d 2 -q
MinimaxAgent | profundidad=2 plies | acción_inicial=South | resultado=limite | finalización=limite de rondas | nodos=954 | nodos_intruso=837 | tiempo=0.018804s
PS C:\Users\nimau\Desktop\Escritorio\Clases 2026-2\IA\IA taller2\taller2_IA> py -3.11 main.py -m adversarial -a MinimaxAgent -l tiny_defense -d 4 -q
MinimaxAgent | profundidad=4 plies | acción_inicial=South | resultado=limite | finalización=limite de rondas | nodos=16438 | nodos_intruso=6755 | tiempo=0.213165s
PS C:\Users\nimau\Desktop\Escritorio\Clases 2026-2\IA\IA taller2\taller2_IA> py -3.11 main.py -m adversarial -a MinimaxAgent -l bottleneck -d 2 -q
MinimaxAgent | profundidad=2 plies | acción_inicial=Stop | resultado=derrota | finalización=intercepción | nodos=27 | nodos_intruso=19 | tiempo=0.000772s
PS C:\Users\nimau\Desktop\Escritorio\Clases 2026-2\IA\IA taller2\taller2_IA> py -3.11 main.py -m adversarial -a MinimaxAgent -l bottleneck -d 4 -q
MinimaxAgent | profundidad=4 plies | acción_inicial=West | resultado=victoria | finalización=activación completa | nodos=2192 | nodos_intruso=1470 | tiempo=0.040579s
```

4.5 Reflexión sobre uso de IA

Usamos IA para entender cómo estructurar el algoritmo Minimax en el juego adversarial de dos agentes con turnos alternados, haciendo que el defensor maximice y el intruso minimice el puntaje. También la usamos para manejar la profundidad por plies, registrar el conteo de estados evaluados en self.nodos_evaluated desde la raíz y aplicar el desempate conservando la primera acción según las reglas del taller.

Prompt usado: Estoy implementando Minimax para el juego del defensor y el intruso. Necesito que el defensor elija el valor más alto y el intruso el más bajo, alternando turnos. La profundidad debe bajar con cada movimiento y quiero contar los nodos que se evalúan, incluida la raíz. ¿Me ayudas a revisar la lógica y los casos terminales?

Punto 5: Poda alfa-beta y funcion de evaluacion:

5.1 Implementación de Poda alfa-beta:

- Incorporamos los parámetros ['a', 'B'] a lo largo de las llamadas recursivas de alpha-beta. 'a' representa el mejor valor garantizado para MAX hasta el momento, mientras que 'B' representa el menor valor garantizado para MIN.

- **Condición MAX** = Al evaluar las ramas sucesoras, si en algún punto se cumple que $v \geq 'B'$ se interrumpe la iteración sobre las acciones restantes en (break). Esto ocurre porque el ancestro MIN ya tiene una alternativa igual o menor a 'B' haciendo imposible que este nodo MAX sea seleccionado.
- **Condición MIN** = Si el valor encontrado $v \leq 'a'$, se poda el resto de las ramas (break). El ancestro MAX ya tiene garantizado un valor de al menos 'a', por lo que descarta cualquier elección que de un resultado inferior.

5.2 evaluation_function: En evaluation_function la función retorna +1000 en victoria y -1000 en derrota conservando las utilidades base, mientras que los estados no terminales se restringen estrictamente en el intervalo [-999, 999] para que al final domine sobre cualquier estimación. Para los cortes por profundidad, combinamos el puntaje acumulado, una penalización por terminales pendientes y por la distancia de seguridad al intruso y por la cantidad de acciones legales disponibles. Además, si la separación con el atacante es de un paso o menor, aplicamos un castigo de 100 puntos para priorizar la evasión inmediata ante una captura.

5.3 Verificación y comparación entre Minimax y Poda Alfa-beta:

	Profundidad	Algoritmo	Acción Inicial	Resultado	Finalización	Nodos
dual_terminal	4	Minimax	East	Victoria	Activación completa	1024
dual_terminal	4	Alfa-Beta	East	Victoria	Activación completa	292
medium_defense	4	Minimax	East	Límite	Límite de rondas	24216
medium_defense	4	Alfa-Beta	East	Límite	Límite de rondas	10981

```
PS C:\Users\nimau\Desktop\Escritorio\Clases 2026-2\IA\IA taller2\taller2_IA>
PS C:\Users\nimau\Desktop\Escritorio\Clases 2026-2\IA\IA taller2\taller2_IA> py -3.11 main.py -m adversarial -a MinimaxAgent -l dual_terminal -d 4 -q
MinimaxAgent | profundidad=4 plies | acción_inicial=East | resultado=victoria | finalización=activación completa | nodos=1024 | nodos_intruso=300 | tiempo=0.016942s
PS C:\Users\nimau\Desktop\Escritorio\Clases 2026-2\IA\IA taller2\taller2_IA> py -3.11 main.py -m adversarial -a AlphaBetaAgent -l dual_terminal -d 4 -q
AlphaBetaAgent | profundidad=4 plies | acción_inicial=East | resultado=victoria | finalización=activación completa | nodos=292 | nodos_intruso=300 | tiempo=0.008365s
PS C:\Users\nimau\Desktop\Escritorio\Clases 2026-2\IA\IA taller2\taller2_IA> py -3.11 main.py -m adversarial -a MinimaxAgent -l medium_defense -d 4 -q
MinimaxAgent | profundidad=4 plies | acción_inicial=East | resultado=límite | finalización=límite de rondas | nodos=24216 | nodos_intruso=9517 | tiempo=0.331644s
PS C:\Users\nimau\Desktop\Escritorio\Clases 2026-2\IA\IA taller2\taller2_IA> py -3.11 main.py -m adversarial -a AlphaBetaAgent -l medium_defense -d 4 -q
AlphaBetaAgent | profundidad=4 plies | acción_inicial=East | resultado=límite | finalización=límite de rondas | nodos=10981 | nodos_intruso=9517 | tiempo=0.211785s
```

En ambos escenarios seleccionaron la misma acción inicial y obtuvieron el mismo desenlace global. Comprobamos que la poda ['a', 'B'] descarta únicamente ramas que matemáticamente no pueden alterar el valor de la raíz, preservando la decisión óptima sin modificar el comportamiento del agente.

- En dual_terminal, Alfa_Beta redujo los nodos explorados por el defensor de 1024 a 292, logrando reducir alrededor del 70% y recortando casi a la mitad el tiempo de ejecución.

- En medium_defense, un mapa de mayor escala y más ramas, los nodos evaluados pasaron de 24216 a 10981, lo que representa una disminución de alrededor del 50%.

El intruso se mantiene idéntico en cada par de experimentos, confirmamos que el entorno adversario respondió de forma determinista y bajo la misma secuencia de juego frente a ambos agentes.

5.4 Justificación de características, pesos y discriminación de estados:

La función de evaluación combina las señales de avance hacia la meta con márgenes de preservación física frente al intruso ponderadas de acuerdo a la prioridad del problema.

- Al restringir cualquier evaluación intermedia al rango [-999, 999] mediante recorte, garantizamos que ningún estado intermedio altere el valor de una victoria asegurada(1000), ni que una posición riesgosa sea descartada con el mismo peso absoluto que una derrota terminal (-1000).
- **Terminales pendientes(-35):** Es el motor principal del agente. Cada vez que activa una terminal, se quita ese castigo de encima, es decir que gana 35 puntos y lo empuja a completar la misión.
- **Distancia a la terminal más cercana:** Al usar la distancia real por los pasillos en (state.layout.distance) y no en línea recta, el defensor sabe que tan lejos está en verdad de su objetivo y no se queda trabado contra paredes o esquinas sin salida.
- **Distancia al intruso (5.0 con tope de 20):** Le da puntos por mantenerse lejos del enemigo. Le pusimos un tope de 20 celdas para que, una vez esté a salvo, deje de huir y se concentre en ir a activar las terminales.
- **Peligro inminente (-100.0 si está a 1 paso):** Si el intruso queda al lado, lo puede atrapar en el siguiente turno. Este castigo de -100 le duele más que cualquier otra cosa, obligándolo a esquivar o retroceder antes que intentar avanzar a una terminal.
- **Movilidad (0.5):** Le da un pequeño empujón por tener más salidas disponibles, evitando meterse en pasillos ciegos donde el intruso lo pueda arrinconar.

Distingue estados así:

- Favorable si el defensor está pegado a una terminal, tiene al intruso a más de 2 pasos y cuenta con varias salidas libres, el puntaje se va arriba y busca esa jugada.
- Desfavorable si el intruso está a un solo paso de atraparlo, le quedan pocas casillas a donde escapar y todavía faltan terminales; el puntaje se desploma y el algoritmo descarta esa rama para no morir.

5.6 Reflexión de la IA:

Usamos IA para entender cómo implementar la poda alfa-beta actualizando y cortando las ramas con ['a', 'B'] sin cambiar la decisión original de Minimax. También la usamos para diseñar la

función de evaluación en evaluation.py, integrando las terminales pendientes, las distancias reales sobre el mapa, el margen de seguridad frente al intruso y el acotamiento de valores no terminales.

Prompt usado: Ya tengo Minimax funcionando. Ahora quiero implementar poda alfa-beta manteniendo el mismo orden de acciones y la misma decisión. También necesito mejorar la función de evaluación para que considere las terminales pendientes, la distancia a la más cercana, la posición del intruso y las opciones de movimiento del defensor. ¿Me ayudas a comparar los nodos explorados por ambos algoritmos?

Punto 6: Analisis algoritmos y resultados:

6.1: Características que explican las diferencias observadas:

- Plateau_grid y local_peak: En nuestras pruebas, ascenso de colina se quedó estancado en solo 2 evaluaciones promedio, obteniendo 61.40 en plateau_grid y 70.00 en local_peak muy lejos del óptimo de 94.00 y 96.00 respectivamente. Esto pasa porque el algoritmo solo acepta mejoras estrictas ante una meseta donde los vecinos tienen igual o menor puntaje, se frena.
- En plateau_grid, el recocido simulado si alcanzó el óptimo de 94.00 porque la probabilidad le permitió aceptar movimientos que bajaban el puntaje temporalmente para atravesar la zona plana y encontrar el camino hacia la cima.
- Al haber muchas combinaciones de módulos, los algoritmos locales tardan más en explorar los extremos opuestos del mapa. El algoritmo genético manejó esta complejidad combinando partes de distintas soluciones con el cruce y la mutación, y así alcanzó puntajes altos de hasta 283 entre las diferentes generaciones.

6.2: Resultados algoritmos de optimizacion:

- Inicialización: En ascenso de colina la solución inicial lo determina todo. Si empieza en la ladera de un pico bajo, no tiene forma de corregir el rumbo, lo que explica la alta desviación estándar que tuvimos.
- Aleatoriedad y estabilidad: En recocido simulado y algoritmos genéticos la aleatoriedad sirve para explorar, pero al promediar 5 ejecuciones se vio que convergen a soluciones muy estables (desviaciones bajas de 0.80 a 2.19).
- Parámetros en recocido simulado: El factor de enfriamiento alfa definió la duración de la búsqueda. Con $\alpha = 0.90$ la temperatura bajó rápido y terminó en 225 evaluaciones, mientras que con $\alpha = 0.99$ se mantuvo caliente más tiempo y completó las 750 iteraciones (751 evaluaciones). Ambos llegaron a 94, pero $\alpha = 0.99$ dio una desviación de 0 frente a 0.80 con $\alpha = 0.90$.
- Parámetros en algoritmo genético:
 - Poblacion: Subir de 20 a 40 individuos subió el promedio de 281.00 a 282.20 y redujo la desviación a más de la mitad (de 2.19 a 0.98), dando mayor consistencia.

- **Mutuacion:** Subir la mutación de 0.05 a 0.20 desestabilizó las soluciones buenas ya encontradas, bajando el promedio a 281.40 y subiendo la desviación a 1.50.
- **Generaciones:** Pasar de 100 a 200 generaciones duplicó el esfuerzo (de 4040 a 8040 evaluaciones) sin aportar mejora en el puntaje máximo (quedó igual en 283), mostrando que la población ya había convergido.

6.3 Compromiso entre calidad de la solución y costo computacional en los algoritmos de optimización:

- **Ascenso de colina:** Costo mínimo, pero mala calidad de solución y alta variabilidad por quedar atrapado en óptimos locales. Sirve si se necesita una respuesta instantánea sin importar la precisión.
- **Recocido simulado:** El de mejor balance para un solo estado. Requiere un costo moderado y logró alcanzar el óptimo de 94.00 de manera repetida.
- **Algoritmo genético:** Calidad muy alta y consistente (promedios sobre 281 en `balanced_city` pero con un costo computacional mucho más elevado. Vale la pena cuando el problema es grande y complejo, pero aumentar generaciones sin control genera un gasto computacional innecesario una vez la población converge.

6.4 Impacto de la profundidad en la decisión del defensor y efecto horizonte:

- En el mapa `bottleneck`, con profundidad $d=2$ el defensor eligió la acción `Stop` y terminó en derrota por intercepción (en 27 nodos). Cuando aumentamos a profundidad 4, cambió su acción inicial a `West` y logró la victoria por activación completa en 2192 nodos.
- Con profundidad 2, el defensor sólo anticipa un movimiento propio y la respuesta inmediata del intruso. La función de evaluación no alcanza a ver que detenerse o dar un paso corto conduce a una celda sin salida en los siguientes turnos. Con profundidad 4, el agente ve la secuencia de dos turnos completos ($MAX \rightarrow MIN \rightarrow MAX \rightarrow MIN$), detecta la emboscada futura y reacciona a tiempo tomando `West` para evitar el callejón y ganar la partida.

6.5: Equivalencia Minimax vs alfa-beta, costo y orden de acciones:

- **Porque toman la misma decisión:** Alfa-beta descarta únicamente ramas donde matemáticamente el valor no podrá superar al mejor ya garantizado (alfa) o donde el adversario ya puede forzar algo peor (beta). Como ninguna rama podada podría haber modificado el valor minimax de la raíz, la decisión tomada es idéntica bajo las mismas condiciones.
- **Impacto profundidad en el costo:** En Minimax el costo crece exponencialmente como $O(b^d)$ donde b es el número de movimientos y d la profundidad. Aumentar plies dispara el tiempo y el número de estados (en `tiny_defense`, pasar de 2 a 4 plies aumentó los nodos de 954 a 16438).

- **Impacto del orden de las acciones en la poda:** La eficiencia de la poda depende de evaluar primero las mejores jugadas. Si las acciones se examinan en el mejor orden posible, establecemos cotas alfa altas y cotas beta bajas, permitiendo cortar subárboles enteros inmediatamente y reduciendo la complejidad.